Automating Tasks with Bash: Hello World & File Listing

Linux Shell Scripting

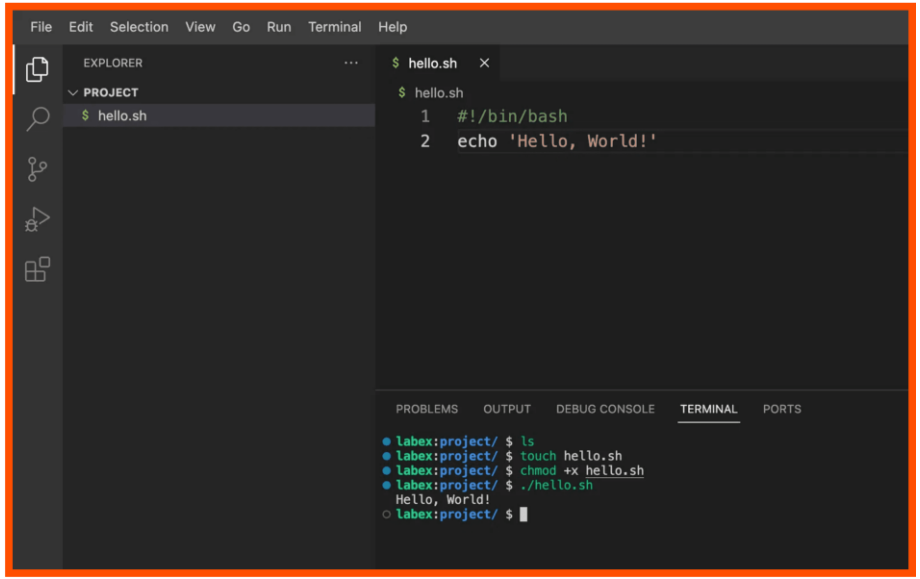
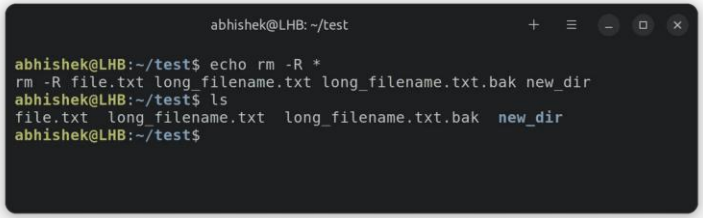


Figure 1. Linux Scripting Environment

What is Bash Scripting?

- **Definition:** A Bash script is a plain text file containing a series of commands for the Linux shell.
- **Automation:** It allows users to automate repetitive tasks by executing multiple commands in sequence.
- **Interpreter:** The script is interpreted by the **Bash** (Bourne Again SHell) program.



Key Benefits

- **Efficiency:** Saves time by automating complex or long command sequences.
- **Repeatability:** Ensures tasks are performed identically every time, reducing human error.
- **System Admin:** Essential for managing servers, backups, and system configurations.

Accessing the Linux Terminal

Preparing Your Environment

- **Open Terminal:** Use the shortcut **Ctrl+Alt+T** or search for "Terminal" in your application menu.
- **Check Directory:** Use the **pwd** command to verify your current working directory.
- **Navigate:** Use **cd** to move to the directory where you want to save your script.
- **Verify Environment:** Ensure you have a text editor like **nano** or **vi** installed.

```
vivek@nixcraft:/tmp$ vi hello.sh
vivek@nixcraft:/tmp$
vivek@nixcraft:/tmp$ chmod +x hello.sh
vivek@nixcraft:/tmp$
vivek@nixcraft:/tmp$ ls -l hello.sh
-rwxr-xr-x 1 vivek vivek 31 Jan 21 15:08 hello.sh
vivek@nixcraft:/tmp$
vivek@nixcraft:/tmp$ ./hello.sh
Hello World
vivek@nixcraft:/tmp$
vivek@nixcraft:/tmp$ bash hello.sh
Hello World
vivek@nixcraft:/tmp$
vivek@nixcraft:/tmp$ sh hello.sh
Hello World
vivek@nixcraft:/tmp$ cat hello.sh
#!/bin/bash
echo "Hello World"
vivek@nixcraft:/tmp$
```

Figure 3. Accessing the Command Line Interface

Writing Your First Bash Script

Creating the Script File

- **Choose an Editor:** For beginners, **nano** is a simple and effective text editor available in most Linux terminals.
- **Create File:** Type the command **nano myscript.sh** to create and open a new file named "myscript.sh".
- **File Extension:** Using the **.sh** extension is a best practice to identify the file as a shell script.
- **Prepare to Code:** Once the editor opens, you are ready to start writing your Bash commands.

Figure 4. Creating a Script File in the Nano Editor

The Shebang Line

Defining the Script Interpreter

```
#!/bin/bash
```

- **Purpose:** The shebang tells the operating system which interpreter to use to execute the script's commands.
- **Placement:** It must be the absolute first line of the file, starting with the **#!** characters.
- **Path:** **/bin/bash** is the standard path for the Bash shell on most Linux systems.
- **Execution:** Without this line, the system might use a different shell, leading to potential errors.

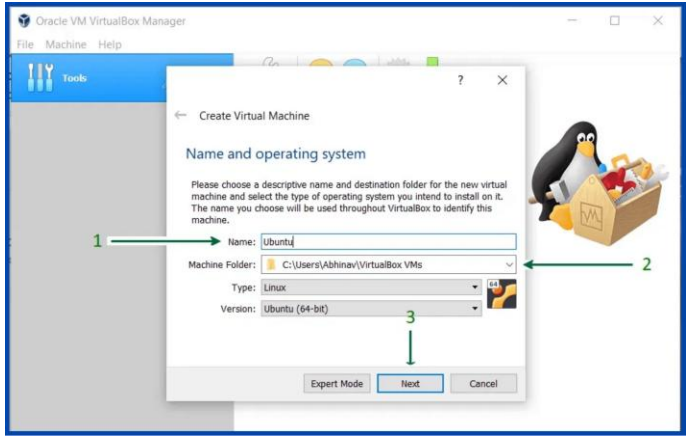


Figure 5. Specifying the Interpreter Environment

Task 1: Printing "Hello, World!"

Using the echo Command

- **The echo Command:** The primary tool for displaying text or variables in the terminal.
- **Syntax:** Simply follow the command with the string you wish to print, enclosed in quotes.

```
echo "Hello, World!"
```

- **Standard Output:** By default, **echo** sends its output to the terminal window.
- **Quotes:** Using double quotes ensures that the string is treated as a single argument.

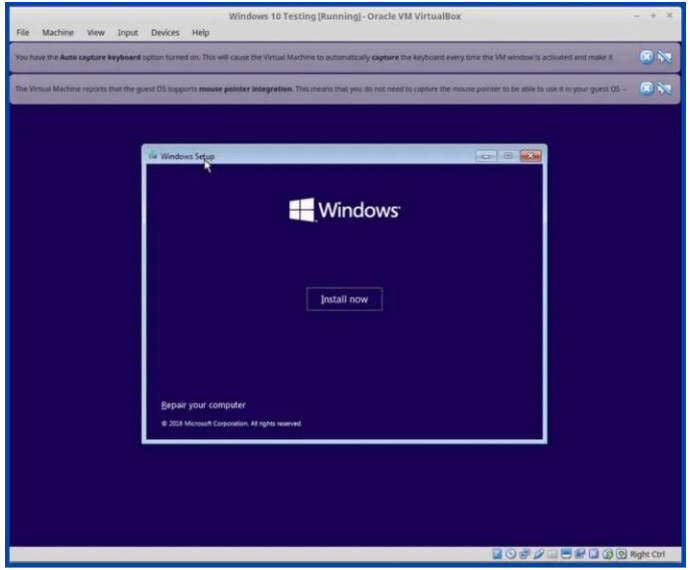


Figure 6. Executing the echo Command

Task 2: Listing Files and Directories

Displaying Directory Contents within a Script

- **The `ls` Command:** The primary command used in Linux to list files and directories in the current working directory.
- **Long Format:** Using `ls -l` provides detailed information, including permissions, owner, size, and modification date.
- **Hidden Files:** The `ls -a` option allows the script to display hidden files (those starting with a dot).
- **Script Integration:** When placed in a script, `ls` executes relative to the directory from which the script is run.
- **Output Formatting:** By default, `ls` outputs to the terminal, providing a clear view of the file system structure to the user.

Combining Tasks: The Complete Script

Finalizing Your Automation Script

```
#!/bin/bash
echo "Hello, World!"
ls
```

- **Save File:** In the nano editor, press **Ctrl+O** and then **Enter** to write the changes to disk.
- **Exit Editor:** Press **Ctrl+X** to close the editor and return to the terminal prompt.
- **Verification:** Your script is now saved as **myscript.sh** and is ready for the next step.

Figure 8. Finalizing the Script in the Text Editor

Granting Execution Permissions

Making Your Script Executable

```
chmod +x myscript.sh
```

- **The chmod Command:** Used to change the access permissions of file system objects.
- **The +x Flag:** Specifically adds the "execute" permission to the file for the user.
- **Security:** By default, new files are not executable to prevent accidental running of malicious code.
- **Verification:** Use **ls -l** to see the file permissions; an 'x' will appear in the permission string.

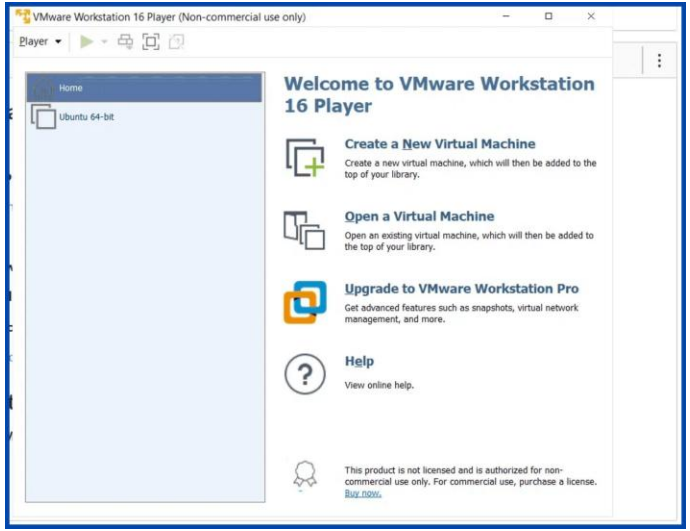


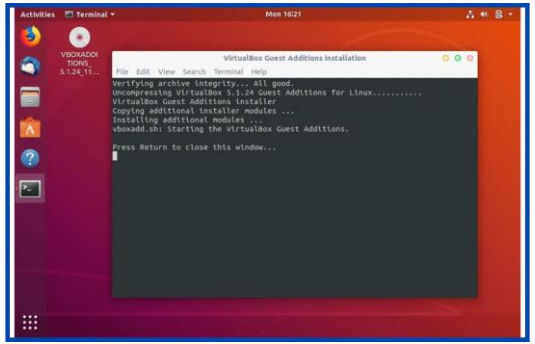
Figure 9. Configuring File Permissions in the Terminal

Running Your Bash Script

Executing and Observing the Output

```
./myscript.sh
```

- **Execution Command:** Use the **./** prefix to tell the shell to look for the script in the current directory.
- **Combined Output:** The terminal will first display "Hello, World!" followed by the list of files and directories.
- **Success Verification:** If the output matches your expectations, your first automation script is a success!



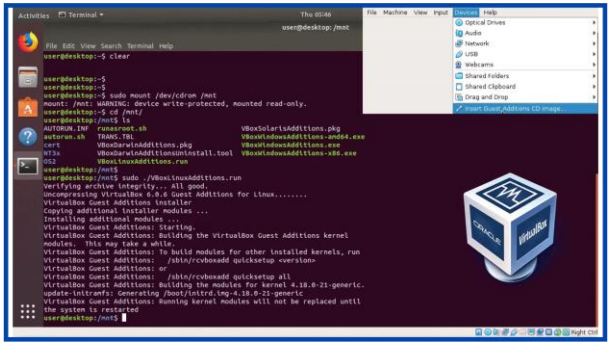


Figure 10. Executing the Script and Verifying Output

Conclusion: Bash Scripting

- **Environment Setup:** Accessing the Linux terminal and navigating the file system is the foundation of scripting.
- **Script Creation:** Using text editors like **nano** to create **.sh** files allows for persistent command storage.
- **Core Components:** The **shebang** specifies the interpreter, while commands like **echo** and **ls** perform the actual work.
- **Permissions & Execution:** Granting execution rights with **chmod** s a critical step before running scripts with **./**.
- **The Power of Automation:** This simple script is the first step toward automating complex system tasks, improving efficiency and reducing errors.